

Chapter 4

Computational Complexity

Introduction

- Even when a problem is decidable and thus computationally solvable in principle, it may not be solvable in practice if the solution requires an inordinate amount of time or memory.
- **Computational complexity theory-is** an investigation of the time, memory, or other resources required for solving computational problems.

- *Algorithmic complexity* is a rough sense of how much computing resources will be required for an algorithm to handle input data of various sizes or values, particularly in terms of time and space (memory or storage.)
- The goal is to derive a mathematical formula which expresses how a change in the size of the input data will impact the overall performance of the algorithm.
- *Computational complexity* is essentially a synonym for *algorithmic complexity*.

- **Computational complexity theory** studies the complexity of algorithms and problems.
- It aims to identify the computational resources required to solve a problem, such as time, space, or communication, and to determine the limitations and possibilities of algorithmic efficiency.

MEASURING COMPLEXITY

Time Complexity

- **Time complexity** is a measure of the amount of time an algorithm takes to run as a function of the size of its input
- It provides an estimate of the number of operations performed by the algorithm relative to the input size.
- Time complexity is typically expressed using big O notation, which describes the upper bound or worst-case scenario for the algorithm's performance.

- The running time of an algorithm purely as a function of the size of the input
 - *Worst-case analysis* : considers the longest running time of all inputs of a particular length.
 - *Average-case analysis*, considers the average of all the running times of inputs of a particular length.

Definition:

- Let M be a deterministic Turing machine that halts on all inputs

The *running time* or *time complexity* of M is the function

$f:N \rightarrow N$, where $f(n)$ is the maximum number of steps that M

uses on any input of length n . If $f(n)$ is the running time of M , we

say that M runs in time $f(n)$ and that M is an $f(n)$ time Turing machine. Customarily we use n to represent the length of the input.

Big-O and Small-o notations

- Because the exact running time of an algorithm often is a complex expression, we usually just estimate it. In one convenient form of estimation, called *asymptotic analysis*, we seek to understand the running time of the algorithm when it is run on large inputs.
- We do so by considering only the highest order term of the expression for the running time of the algorithm, disregarding both the coefficient of that term and any lower order terms, because the highest order term dominates the other terms on large inputs.

- For example, the function $f(n) = 6n^3 + 2n^2 + 20n + 45$ has four terms, and the highest order term is $6n^3$. Disregarding the coefficient 6, we say that f is asymptotically at most n^3 . The *asymptotic notation* or ***big-O notation*** for describing this relationship is $f(n) = O(n^3)$.

Definition

- **Let f and g be functions.** $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ We say $f(n) = O(g(n))$ if positive integers c and n_0 exist such that for every integer $n > n_0$ $f(n) < cg(n)$.
- When $f(n) = O(g(n))$ we say that $g(n)$ is an *upper bound* for $f(n)$, or more precisely, that $g(n)$ is an *asymptotic upper bound* for $f(n)$, to emphasize that we are suppressing constant factors.

Example:

- Let $f(n)$ be the function $5n^3 + 2n^2 + 22n + 6$. Then, selecting the highest order term $5n^3$ and disregarding its coefficient 5 gives $\mathbf{f}(n) = O(n^3)$.

Here are some common time complexity classes:

- **Constant Time ($O(1)$):** The algorithm takes a constant amount of time to execute, regardless of the input size. For example, accessing an element in an array or performing a simple arithmetic operation.
- **Linear Time ($O(n)$):** The algorithm's running time grows linearly with the input size. For example, iterating through each element in a list or performing a linear search.

- **Logarithmic Time ($O(\log n)$):** The algorithm's running time grows logarithmically with the input size. This typically occurs in algorithms that divide the input in half at each step, such as binary search on a sorted list.
- **Polynomial Time ($O(n^k)$):** The algorithm's running time is a polynomial function of the input size, where k is a constant. Examples include algorithms with nested loops, where the number of iterations is determined by a polynomial expression.

Exponential Time ($O(2^n)$):

- The algorithm's running time grows exponentially with the input size. This occurs in algorithms that generate all possible subsets or permutations of the input.

Big O Notation Order

- Here are, once again, the described complexity classes, sorted in ascending order of complexity (for sufficiently large values of n):
 1. $O(1)$ – constant time
 2. $O(\log n)$ – logarithmic time
 3. $O(n)$ – linear time
 4. $O(n \log n)$ – quasilinear time
 5. $O(n^2)$ – quadratic time

Space complexity

- Is a measure of the amount of memory or space required by an algorithm to solve a problem as a function of the input size.
- It provides an estimate of the maximum amount of memory used by the algorithm relative to the input size.
- Similar to time complexity, space complexity is often expressed using big O notation.

Deterministic Algorithm

- In deterministic algorithm for a particular input the computer will give same output every time
- Examples are finding odd or even, sorting, finding max etc
- Most of the algorithms are deterministic in nature

Non-deterministic Algorithm

- In non-deterministic algorithm for a particular input the computer will give different output on different executions
- This algorithm operates in two phases :
 - Guessing and Verification.
- **Randomly picking** some elements from the list and check if it is maximum is non-deterministic.

Difference between Deterministic and Non deterministic Algorithms

Deterministic Algorithm

- For a particular input the computer will give the same output on different executions
- Can solve the problem in polynomial time.

Non-Deterministic Algorithm

- For a particular input the computer will give different outputs on different executions
- Can't solve the problem in polynomial time.

Classifying Problems

- We have seen that decision problems (and their associated languages) can be classified into **decidable** and **undecidable**.
- This result was obtained by Turing and others in the 1930' — before the invention of computers
- After the invention of computers, it became clear that it would be useful to classify **decidable problems**, to distinguish **harder** problems from **easier** problems
- This led to the development of **computational complexity** theory in the 1960's and 1970's

Complexity classes

- categorize computational problems based on the amount of resources, such as time and space, required to solve them.
- These classes provide a way to compare the difficulty of different problems and analyze the efficiency of algorithms.

Here are some commonly encountered complexity classes:

1. P (Polynomial Time)

- This class consists of decision problems (problems with a “yes” or “no” answer)that can be solved by a deterministic Turing machine in polynomial time.
- P is the class of languages that are decidable in polynomial time on a deterministic single-tape Turing machine.
- In other words, there exists an algorithm that can solve these problems efficiently.
- When a problem is in P, we have a method of solving it that runs in time n^k for some constant k .
- Problems in P are considered tractable.
- P roughly corresponds to the class of problems that are realistically solvable on a computer.
- Tractable means that the problems can be solved in theory as well as in practice. But the problems that can be solved in theory but not in practice are known as intractable.

Features:

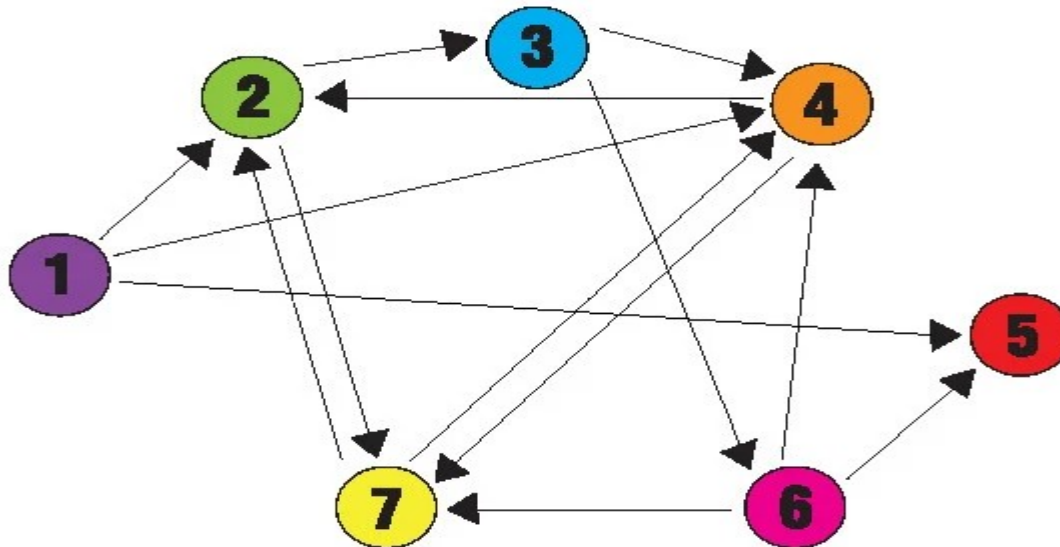
- The solution to **P problems** is easy to find.
- **P** is often a class of computational problems that are solvable and tractable.
- **This class contains many problems:**
 - Calculating the GCD
 - Merge Sort
 - Binary Search
 - Linear Search
 - Insertion sort

2. NP Class Problem:

- Attempts to avoid brute force in certain other problems, haven't been successful, and polynomial time algorithms that solve them aren't known to exist.
- NP is short for nondeterministic polynomial time, since the decision problem in NP are precisely the problems that can be solved on a nondeterministic Turing machine in polynomial time.
- NP does not stand for “not P”, as there are many problems that cannot even be verified in polynomial time.
- A problem is said to be Nondeterministic polynomial time that can be solvable in polynomial time by a nondeterministic Turing machine.
- Problems of NP can be verified by a Turing machine in polynomial time.
- The solutions to the NP class problem are hard to find since they are being solved by a non-deterministic machine.
- Every problem in this class can be solved in exponential time using exhaustive search.

- **Example of NP class problem: A Hamiltonian path Problem**

- The Hamiltonian Path Problem asks whether there is a route in a directed graph from a beginning node to an ending node, visiting each node exactly once.



- There is an exponential time algorithm for the *HAMPATH* problem

- The Hamiltonian Path Problem is to start at node 1, end at node 5, and visit each node exactly once (1→4→7→2→3→6→5).

- **Solving a problem:** Is there a Hamiltonian cycle (or path) in a graph G ?
- **Verifying a candidate solution:** Is $v_0, v_1, \dots, v_l$ a Hamiltonian cycle of graph G ?
- The *HAMPATH* problem does have a feature called *polynomial verifiability*
- ***Other examples of class NP problems include:***
 - The traveling salesman problem, satisfiability problems, and graph-covering problems

Solving vs. Verifying

- Intuitively it seems much harder (more time consuming) in some cases to solve a problem from scratch than to verify that a candidate solution actually solves the problem.
- If there are many candidate solutions to check, then even if each individual one is quick to check, overall it can take a long time

A nondeterministic machine for the HAMPATH

$N1$ = "On input (G, s, t) , where G is a directed graph with nodes s and t :

1. Write a list of m numbers, $p_1, \dots, p_m$, where m is the number of nodes in G . Each number in the list is nondeterministically selected to be between 1 and m .
2. Check for repetitions in the list. If any are found, *reject*.
3. Check whether $s = p_1$ and $t = p_m$. If either fail, *reject*.
4. For each i between 1 and $m - 1$, check whether (p_i, p_{i+1}) is an edge of G . If any are not, *reject*. Otherwise, all tests have been passed, so *accept*."

Features:

- The solutions of the NP class are hard to find since they are being solved by a non-deterministic machine but the solutions are easy to verify.
- Problems of NP can be verified by a Turing machine in polynomial time.

Differences between the P class problem and the NP class problem

Sr. No	P class Problem	NP class Problem
1.	P problems are a set of problems that can be solved in polynomial time by deterministic algorithms.	NP problems are problems that can be solved in nondeterministic polynomial time.
2.	P Problems can be solved and verified in polynomial time.	The solution to NP problems cannot be obtained in polynomial time, but if the solution is given, it can be verified in polynomial time.
3.	P problems are a subset of NP problems.	NP Problems are a superset of P problems.
4.	All P problems are deterministic in nature.	All the NP problems are non-deterministic in nature.

Differences between the P class problem and the NP class problem(Cont'd)

5. It takes polynomial time to solve a problem like n , n^2 , $n \log n$, etc.

It takes non-deterministic polynomial time to quickly check a problem.

6. The solution to P class problems is easy to find.

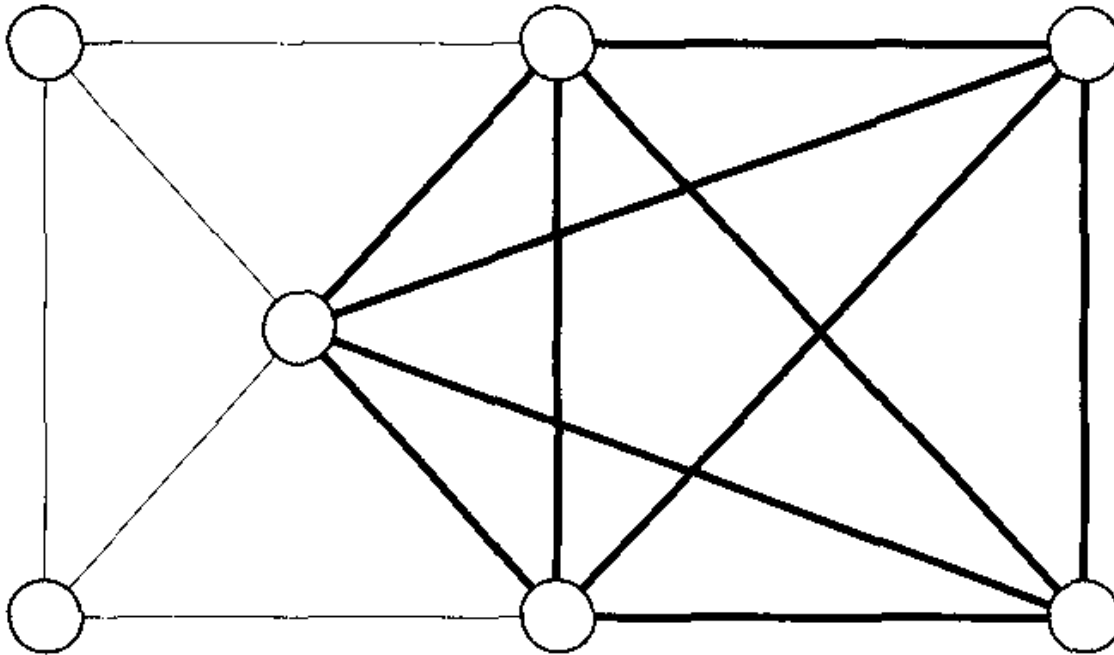
The solution to NP class problems is hard to find.

7. **Examples of P problems are:**
Selection sort, Linear Search

Examples of NP problems are the
Travelling salesman problem and the knapsack problem

Example of Problem in NP

- A *clique* in an undirected graph is a subgraph, wherein every two nodes are connected by an edge. A *k-clique* is a clique that contains *k* nodes.



A graph with a 5-clique

The clique problem

- The clique problem is to determine whether a graph contains a clique of a specified size.
- Let $\text{CLIQUE} = \{ (G, k) \mid G \text{ is an undirected graph with a } k\text{-clique} \}$.
- Theorem : CLIQUE is in NP.

Theorem : Clique is in NP

- **Proof Idea:** The clique is the certificate.

Proof: The following is a verifier V for *CLIQUE*.

$V =$ "On input $((G, k), c)$:

1. Test whether c is a set of k nodes in G
2. Test whether G contains all edges connecting nodes in c .
3. If both pass, *accept*; otherwise, *reject*."

Alternative Proof:

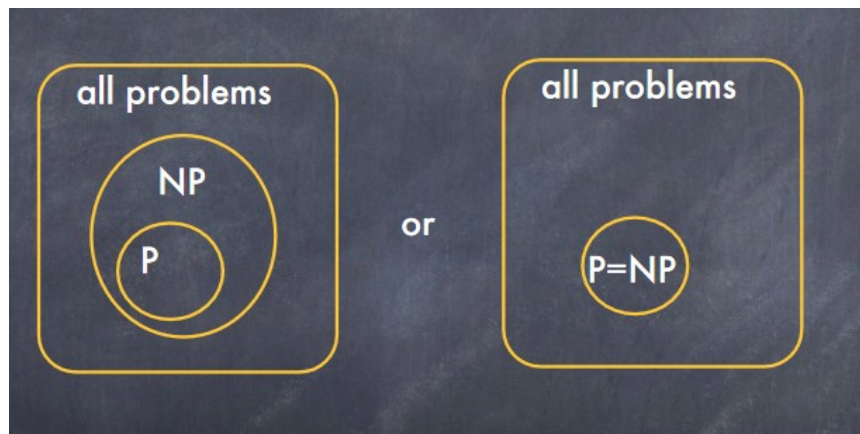
- If you prefer to think of NP in terms of nondeterministic polynomial time Turing machines, you may prove this theorem by giving one that decides *CLIQUE*. Observe the similarity between the two proofs.

$N =$ "On input (G, k) , where G is a graph:

1. Nondeterministically, select a subset c of k nodes of G .
2. Test whether G contains all edges connecting nodes in c .
3. If yes, *accept*; otherwise, *reject*."

THE P VERSUS NP QUESTION

- The question of whether $P = NP$ is one of the greatest unsolved problems in theoretical computer science and contemporary mathematics.
- Most researchers believe that the two classes are not equal because people have invested enormous effort to find polynomial time algorithms for certain problems in NP, without success.
- The following figure shows the two possibilities.



So it is unknown
whether
 $P = NP$.

3. NP-complete (Nondeterministic Polynomial Time Complete)

A decision problem L is NP-complete if:

1. L is in NP (Any given solution to NP-complete problems can be verified quickly, but there is no efficient known solution)
2. Every problem in NP is reducible to L in polynomial time
 - If a problem is NP and all other NP problems are polynomial-time reducible to it, the problem is NP-complete.
 - This is a subset of NP where every problem in NP can be reduced to any NP-complete problem in polynomial time.

NP-complete(Cont'd)

- NP-complete problems are a subset of the larger class of NP
- NP-complete problems are the hardest problems in the NP set.
- If a **problem is NP** and all other NP problems are polynomial-time reducible to it, the problem is NP-complete.
- This is a subset of NP where every problem in NP can be reduced to any NP-complete problem in polynomial time.

NP-complete(Cont'd)

- Thus, finding an efficient algorithm for any NP-complete problem implies that an efficient algorithm can be found for all such problems
- If a polynomial-time algorithm can be found for any NP-complete problem, it would imply that $P = NP$.
- **NP-complete problems are considered to be among the hardest problems in NP.**

Examples:

- Knapsack problem
- Hamiltonian path problem
- Vertex cover problem
- Boolean satisfiability problem
- Clique problem.

Definition:

- A function $f: \Sigma^* \rightarrow \Sigma^*$ is a *polynomial time computable function* if some polynomial time Turing machine M exists that halts with just $f(w)$ on its tape, when started on any input w .

Definition:

Language A is Polynomial time mapping reducible, or simply *polynomial time reducible*, to language B, written $A \leq_p B$, if a polynomial time computable function $f: \Sigma^* \rightarrow \Sigma^*$ exists, where for every w ,

$$w \in A \quad \leftrightarrow \quad f(w) \in B.$$

The function f is called the *polynomial time reduction* of A to B.

THEOREM

- If B is NP-complete and $B <_p C$ for C in NP, then C is NP-complete.

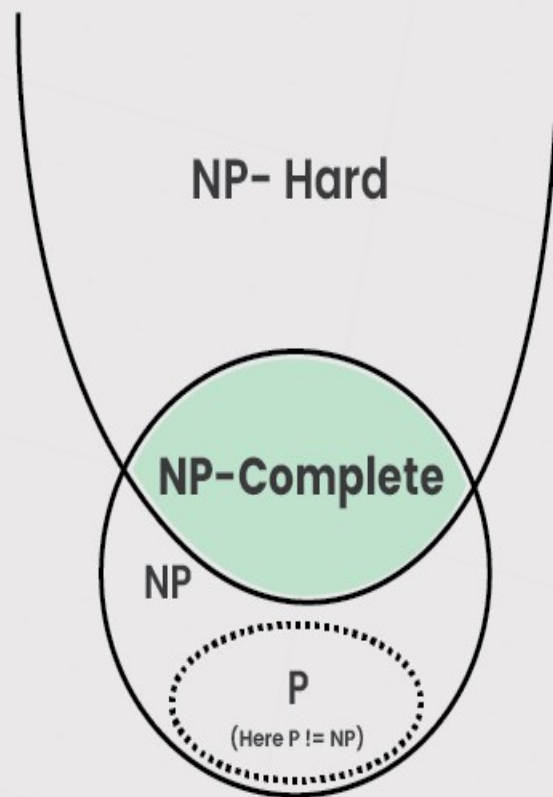
4. NP-hard (Nondeterministic Polynomial Time Hard):

- This class consists of decision problems that are at least as hard as the hardest problems in NP. Unlike NP-complete problems, NP-hard problems may not be in NP themselves.
- **A problem L is NP-Hard if every problem in NP is reducible to L in polynomial time**
- Therefore, a problem is NP-complete if it is both NP and NP-hard
- **NP-Complete set is also a subset of NP-hard set.**



NP-COMPLETE

Complexity Classes



- A problem X is NP-hard, if there is an NP-complete problem Y , such that Y is reducible to X in polynomial time.

Example

- The **halting problem** is an NP-hard problem. This is the problem that given a program P and input I , will it halt? This is a decision problem but it is not in NP. It is clear that any NP-complete problem can be reduced to this one.

5. EXP (Exponential Time)

- This class consists of decision problems that can be solved by a **deterministic Turing machine in exponential time.**
- These problems are significantly harder than problems in P and are considered to be intractable.

6.PSPACE (Polynomial Space)

- This class consists of decision problems that can be solved by a **deterministic Turing machine** using a polynomial amount of memory space.
- **PSPACE** includes problems that can be **solved in polynomial time as well as problems that require exponential time** but **only polynomial space**.

How to prove that a given problem is NP-complete?

- By definition, it requires us to show every problem in NP is polynomial time reducible to L.
- There is an alternate way to prove it. The idea is to take a **known NP-Complete problem** and **reduce it to L**.
- If a polynomial-time reduction is possible, we can prove that L is NP-Complete by transitivity of reduction (If an NP-Complete problem is reducible to L in polynomial time, then all problems are reducible to L in polynomial time).

The Satisfiability Problem

- The **first NP-Complete problem** proved by the definition of NP-Complete problems.
- The Satisfiability Problem in its most general form is determining whether it is possible to satisfy a given set of logical constraints
- Given a formula for Boolean logic, is there any assignment of T/F to its variables that makes the entire formula true?
- Could take many forms:
 - Boolean expressions & logic circuits
- Was the original NP-Complete Problem
 - Shown in 1970 by Stephen Cook

Boolean Expressions

- Boolean, or propositional-logic expressions are built from variables and constants using the operators AND, OR, and NOT.

Example: Boolean expression

$$(A \vee B) \wedge (B \vee \neg C) \wedge (A \vee \neg B)$$

Conjunctive Normal Form

- A Boolean formula is in *Conjunctive Normal Form* (CNF) if it is the AND of *clauses*.
- Each clause is the OR of *literals*.
- A literal is either a variable or the negation of a variable.

3-Satisfiability (3SAT)

- **Input:** A collection of clauses C where each clause contains exactly 3 literals, over a set of Boolean variables v .
- **Output:** Is there is a truth assignment to V such that each clause is satisfied?

CNF-SAT Problem

- is a Boolean formula in CNF satisfiable?

Example: $(A \vee B \vee \neg C) \wedge (B \vee \neg C) \wedge (A \vee \neg B \vee C)$

- Could easily use an exhaustive search in the form of a truth table to solve this problem

THE COOK-LEVIN THEOREM

Theorem : SAT is NP-complete.